

```

import os
import logging
from datetime import datetime, time as dttime, timezone, timedelta
from telegram import Update, InlineKeyboardButton, InlineKeyboardMarkup
from telegram.ext import Application, CommandHandler, CallbackQueryHandler, Conte
from tips_engine import get_best_tip, get_daily_summary, run_diagnostic
from results_engine import get_results, build_results_summary

logging.basicConfig(format="%(asctime)s | %(levelname)s | %(message)s", level=log
logger = logging.getLogger(__name__)

BOT_TOKEN = os.environ.get("BOT_TOKEN", "").strip()
CHANNEL_ID = os.environ.get("CHANNEL_ID", "").strip()
ADMIN_ID = int(os.environ.get("ADMIN_ID", "0").strip())

if not BOT_TOKEN:
    raise RuntimeError("BOT_TOKEN environment variable is not set.")
if not CHANNEL_ID:
    raise RuntimeError("CHANNEL_ID environment variable is not set.")

EST = timezone(timedelta(hours=-5))

sent_today = {"date": "", "tip_sent": False, "tip": None, "sent_id": ""}
scheduled_tip = {}

def reset_if_new_day():
    today = datetime.now(EST).strftime("%Y-%m-%d")
    if sent_today["date"] != today:
        sent_today["date"] = today
        sent_today["tip_sent"] = False
        sent_today["tip"] = None
        sent_today["sent_id"] = ""
        scheduled_tip.clear()

def tip_card(tip):
    prob = str(tip.get("win_prob", "?")) + "%"
    bar_filled = int(tip.get("win_prob", 0) / 10)
    bar = "#" * bar_filled + "-" * (10 - bar_filled)
    return "\n".join([
        "=====",
        "TIP OF THE DAY",
        "====="
    ])

```

```

        tip["match"],
        tip["date"] + " | " + tip["league"],
        "-----",
        "Tip: " + tip["tip"],
        "Odds: " + str(tip["odds"]),
        "Win Probability: " + prob,
        "Confidence: [" + bar + "]" + prob,
        tip["reasoning"],
        "KICKOFF IN 1 HOUR",
        "-----",
        "Gamble responsibly. 18+",
    ])

```

```

def tip_id(tip):
    return tip["match"] + "|" + tip["tip"]

```

```

async def send_tip_job(ctx: ContextTypes.DEFAULT_TYPE):
    reset_if_new_day()
    tip = ctx.job.data
    if sent_today["tip_sent"]:
        return
    tid = tip_id(tip)
    if sent_today["sent_id"] == tid:
        return
    try:
        await ctx.bot.send_message(CHANNEL_ID, tip_card(tip))
        sent_today["tip_sent"] = True
        sent_today["tip"] = tip
        sent_today["sent_id"] = tid
        logger.info("Tip sent: %s", tid)
    except Exception as e:
        logger.error("Failed to send tip: %s", e)

```

```

async def daily_scan(ctx: ContextTypes.DEFAULT_TYPE):
    reset_if_new_day()
    if sent_today["tip_sent"]:
        logger.info("Tip already sent today.")
        return
    try:
        tip = await get_best_tip()
    except Exception as e:
        logger.error("Scan failed: %s", e)
        return
    if not tip:

```

```

        logger.info("Scan: no qualifying tip found.")
        return
    tid = tip_id(tip)
    now = datetime.now(timezone.utc)
    kickoff = None
    try:
        ks = tip.get("kickoff_utc", "")
        kickoff = datetime.fromisoformat(ks.replace("Z", "+00:00"))
        if kickoff.tzinfo is None:
            kickoff = kickoff.replace(tzinfo=timezone.utc)
    except Exception:
        pass
    if kickoff and kickoff > now:
        send_time = kickoff - timedelta(hours=1)
        if send_time <= now:
            ctx.job_queue.run_once(send_tip_job, when=5, data=tip, name=tid)
            logger.info("Tip firing immediately (within 1hr of kickoff): %s", tid)
        else:
            secs = (send_time - now).total_seconds()
            ctx.job_queue.run_once(send_tip_job, when=secs, data=tip, name=tid)
            scheduled_tip["tip"] = tip
            scheduled_tip["send_time"] = send_time
            logger.info("Tip scheduled at %s EST", send_time.astimezone(EST).strftime("%p %I:%M %a"))
        else:
            ctx.job_queue.run_once(send_tip_job, when=5, data=tip, name=tid)
            logger.info("Tip firing immediately (no kickoff time): %s", tid)

async def start(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    kb = [
        [InlineKeyboardButton("Today's Tip", callback_data="tips")],
        [InlineKeyboardButton("Daily Summary", callback_data="summary")],
        [InlineKeyboardButton("Today's Results", callback_data="results")],
        [InlineKeyboardButton("How It Works", callback_data="howto")],
    ]
    await update.message.reply_text(
        "Value Bet Bot\n\n1 tip per day.\nScans at 6am EST for the best odds betw
        reply_markup=InlineKeyboardMarkup(kb),
    )

async def tips_command(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    msg = await update.message.reply_text("Fetching best tip...")
    try:
        tip = await get_best_tip()
    except Exception as e:
        await msg.edit_text("Error: " + str(e))

```

```

        return
    if not tip:
        await msg.edit_text("No tips in the 1.30-1.50 range right now.")
        return
    await msg.delete()
    await update.message.reply_text(tip_card(tip))

async def summary_command(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    try:
        text = await get_daily_summary()
    except Exception as e:
        text = "Error: " + str(e)
    reset_if_new_day()
    if scheduled_tip.get("send_time"):
        text += "\nScheduled for: " + scheduled_tip["send_time"].astimezone(EST).
    elif sent_today["tip_sent"]:
        text += "\nTip sent today."
    else:
        text += "\nNo tip scheduled yet. Scan runs at 6am EST."
    await update.message.reply_text(text)

async def results_command(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    msg = await update.message.reply_text("Checking results...")
    reset_if_new_day()
    if not sent_today["tip"]:
        await msg.edit_text("No tip sent today yet.")
        return
    try:
        tip_results = await get_results([sent_today["tip"]])
        summary = build_results_summary(tip_results, datetime.now(EST).strftime("%
        await msg.edit_text(summary)
    except Exception as e:
        await msg.edit_text("Error: " + str(e))

async def diagnose_command(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    msg = await update.message.reply_text("Running diagnostics...")
    try:
        report = await run_diagnostic()
    except Exception as e:
        await msg.edit_text("Diagnostic error: " + str(e))
        return
    await msg.edit_text(report)

```

```

async def button(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()
    if query.data == "tips":
        try:
            tip = await get_best_tip()
        except Exception as e:
            await query.edit_message_text("Error: " + str(e))
            return
        if not tip:
            await query.edit_message_text("No tips in the 1.30-1.50 range right n
            return
        await query.edit_message_text(tip_card(tip))
    elif query.data == "summary":
        await query.edit_message_text(await get_daily_summary())
    elif query.data == "results":
        reset_if_new_day()
        if not sent_today["tip"]:
            await query.edit_message_text("No tip sent today yet.")
            return
        try:
            tip_results = await get_results([sent_today["tip"]])
            summary = build_results_summary(tip_results, datetime.now(EST).strftime
            await query.edit_message_text(summary)
        except Exception as e:
            await query.edit_message_text("Error: " + str(e))
    elif query.data == "howto":
        await query.edit_message_text(
            "How It Works\n\n"
            "Every day at 6am EST the bot scans all sports for the single best ti
            "The tip is sent to the channel exactly 1 hour before kickoff.\n\n"
            "At 11pm EST a results summary is posted showing WIN or LOSS.\n\n"
            "1 scan per day keeps usage within the free API tier.\n\n"
            "Gamble responsibly."
        )

```

```

async def post_results_summary(ctx: ContextTypes.DEFAULT_TYPE):
    reset_if_new_day()
    scheduled_tip.clear()
    if not sent_today["tip"]:
        logger.info("Results: no tip sent today.")
        return
    try:
        tip_results = await get_results([sent_today["tip"]])
        summary = build_results_summary(tip_results, datetime.now(EST).strftime("
        await ctx.bot.send_message(CHANNEL_ID, summary)
    
```

```

        logger.info("Results posted.")
    except Exception as e:
        logger.error("Results failed: %s", e)

async def push(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    if update.effective_user.id != ADMIN_ID:
        await update.message.reply_text("Admin only.")
        return
    await update.message.reply_text("Triggering daily scan...")
    await daily_scan(ctx)
    if scheduled_tip.get("send_time"):
        t = scheduled_tip["send_time"].astimezone(EST).strftime("%H:%M EST")
        await update.message.reply_text("Done. Tip scheduled for " + t)
    elif sent_today["tip_sent"]:
        await update.message.reply_text("Done. Tip sent to channel.")
    else:
        await update.message.reply_text("Done. No qualifying tip found right now.")

def main():
    app = Application.builder().token(BOT_TOKEN).build()
    app.add_handler(CommandHandler("start", start))
    app.add_handler(CommandHandler("tips", tips_command))
    app.add_handler(CommandHandler("summary", summary_command))
    app.add_handler(CommandHandler("results", results_command))
    app.add_handler(CommandHandler("push", push))
    app.add_handler(CommandHandler("diagnose", diagnose_command))
    app.add_handler(CallbackQueryHandler(button))

    # Daily scan at 6am EST = 11:00 UTC
    app.job_queue.run_daily(daily_scan, time=datetime(hour=11, minute=0))

    # Results summary at 11pm EST = 04:00 UTC
    app.job_queue.run_daily(post_results_summary, time=datetime(hour=4, minute=0))

    logger.info("Bot running. 1 scan at 6am EST. Tip sent 1 hour before kickoff.")
    app.run_polling(drop_pending_updates=True)

if __name__ == "__main__":
    main()

```